

Functional & Non-Functional Requirements (FR/NFR) Document

Project: “SmartTasks” – A Simple To-Do List Web App

Purpose: Manage task lists, a basic Jira implementation

Training Modes:

1. **Solo Mode** – each participant codes individually with Copilot assistance.
2. **Pair Mode** – two participants collaborate, using Copilot as a "third teammate".
3. **Guided Mode** – trainer provides checkpoints, participants use Copilot to fill in code.

1. Project Overview

“SmartTasks” is a lightweight to-do list web application that allows users to create, update, and manage daily tasks.

The app should support user authentication, task management, and basic analytics (like total completed tasks).

2. Functional Requirements (FRs)

ID	Requirement	Description	Acceptance Criteria
F R-	User Registration &	Users can sign up and log in using email and password.	- Can register and log in successfully.
F R-	Add New Task	User can add a new task with title and optional description.	- Task is saved in DB and shown in task list.
F R-	Mark Task Complete	User can mark a task as completed or reopen it.	- UI updates and DB reflects completion state.
F R-	Delete Task	User can delete a task permanently.	- Task is removed from DB and not visible in UI.
F R-	View All Tasks	User can see all their tasks, filtered by status (All / Completed / Pending).	- Tasks display with correct filters applied.
F R-	View Stats	User can see total tasks and completed tasks count.	- Counts update dynamically based on DB records.

3. Non-Functional Requirements (NFRs)

ID	Categor	Requirement	Description
NF R-	Perform ance	The app should load within 2 seconds for 90% of users.	Optimize API and database queries.
NF R-	Security	User passwords must be stored using hashed encryption .	Use libraries like bcrypt or argon2.
NF R-	Usabilit y	The interface should be simple and responsive .	Support desktop and mobile viewports.
NF R-	Reliabili ty	The app should handle graceful errors (e.g., network issues).	Display appropriate error messages.
NF R-	Scalabili ty	Should support up to 1,000 concurrent users without performance degradation.	Use pagination and API rate limits.
NF R-	Maintai nability	Code should follow consistent linting and formatting standards.	Implement ESLint/Black, and maintain modular architecture.
NF R-	Observa bility	Include basic logging for key actions (user login, task changes).	Logs stored locally or in console output.

4. Suggested Tech Stack

Layer	Technology
Frontend	React / Next.js
Backend	Node.js (Express) or Python (FastAPI)
Database	SQLite / PostgreSQL
Auth	JWT-based authentication
Dev Tools	GitHub Copilot, VS Code, Docker (optional)

5. Implementation Plan by Training Mode

Mode	Focus Area	Suggested Copilot Prompts
Solo Mode	Full-stack CRUD implementation	“Generate an Express route for adding a new task with validation.”
Pair Mode	API + Frontend integration	“Suggest a React component for displaying a filtered list of tasks.”
Guided Mode	Incremental feature building	“Add user authentication to FastAPI app using JWT tokens.”

6. Sample Checkpoints

1. Setup project repo and dependencies

2. Create API routes for tasks
3. Connect frontend to backend
4. Add authentication and JWT
5. Style and make responsive
6. Add analytics and logging

7. Expected Outcome

By the end of the session, participants should:

- Understand how to **translate FR/NFR into implementation tasks**
- Use **Copilot prompts effectively** to generate meaningful code
- Collaborate efficiently in different **training modes**
- Deliver a functional “SmartTasks” prototype meeting defined FR/NFR goals